A Constraint-Based High School Scheduling System

Masazumi Yoshikawa, Kazuya Kaneko, Toru Yamanouchi, and Masanobu Watanabe
C&C Research Laboratories, NEC Corporation

CONSTRUCTING ANNUAL CLASS SCHEDULES FOR A TYPICAL JAPANESE HIGH SCHOOL CONSUMES HUNDREDS OF HOURS. THE WINDOWS-BASED SCHOOLMAGIC SCHEDULING SYSTEM USES AN ARC-CONSISTENCY ALGORITHM TO GENERATE HIGH-QUALITY INITIAL ASSIGNMENTS, WHICH IT THEN REFINES WITH A HILL-CLIMBING ALGORITHM TO REDUCE SCHEDULING TIME 50-FOLD.

AT THE START OF EACH NEW school year, schools everywhere must undertake the laborious, time-consuming process of slotting students, teachers, and lessons into available classrooms. Timetable scheduling at Japan's Saitama Prefectural Kuki-Hokuyou High School, for instance, takes 100 person-days per year. At the Kouchi Prefectural Okou High School, where scheduling starts at the beginning of April and doesn't finish until June—two months after classes have begun—the figure rises to 150 person-days.

Quite naturally, schedule-making educators yearn for a simpler, more automatic system. So far, however, the problem has eluded such approaches. For instance, the schedule maker at Daito Bunka University's Dai-Ichi High School still needs 12 or more days to complete a schedule using a commercial scheduling software package. (See the sidebar on the next page for a description of related school scheduling research.)

To speed this process, we first developed a Lisp prototype of a general-purpose *constraint relaxation problem (CRP) solver* called *Coastool* (Constraint-Based Assignment and Scheduling Tool). Coastool solves problems merely by declaring "what the problem is," without programming "how to solve it." Its novel problem-solving method uses an arc-consistency algorithm to generate a high-quality initial assignment, then refines the solution with a hill-climbing algorithm. In our tests with actual high school scheduling problems, Coastool dramatically decreased scheduling costs.[1]

Moreover, by reimplementing Coastool with C++, we have developed the Windows-based SchoolMagic automatic high school scheduling system. By using *virtual constraints* to minimize computational time and memory size, SchoolMagic reduced scheduling costs at the largest of several Japanese high schools we studied to three person-days, for a 50-fold decrease.

School scheduling problem

A high school timetable scheduling problem consists of fundamental data—classes or groups of students, subjects, teachers, rooms, and lessons—and various constraints with penalty points. The challenge is to assign a weekly time slot to every lesson, while minimizing the total violated-constraint penalty. Constraints include

- no class and no teacher can have more than one lesson at a time,
- room capacities must be observed,
- a set of lessons must be taken simultaneously or continuously,
- a class should have no more than one lesson on a subject in a day,
- a part-time teacher or a room has unavailable time slots,
- a teacher has limits on the number of all lessons and continuous lessons in a day,
- two-credit lessons should be taken two days or more apart by a class.

We can formalize the problem as a CRP, which involves variables and constraints that are associated with each other. The domain of a variable has a set of applicable values.

0885-9000/96/$4.00 © 1996 IEEE

Related work

School time-table scheduling has been studied for the last three decades, on the basis of Operations Research techniques.[1] Because an OR technique depends on a simple problem definition, it might fail to solve real-world complex problems. Genetic algorithm and backtracking approaches are also available.[2,3]

Most previous studies focused on university student or examination scheduling, in which a student may have free time in a scheduling period. The constraints in high school scheduling are much tighter, because a student has no free time during the week. We do not know of any successful work on practical high school scheduling problems.

As with the Coastool architecture, constraint logic programming languages provide declarative frameworks to describe and solve a problem. Our previous system and Multi-Tac also provide a declarative framework and a problem-solving program synthesizing facility.[4,5]

For the problem-solving method, Minton and his colleagues developed an initialization method for *Graph Coloring Problems* using problem-specific ordering heuristics.[6] The basic idea may be similar to the RFLG algorithm.

References

1. M.W. Carter, "A Survey of Practical Applications of Examination Scheduling Algorithms," *Operations Research*, Vol. 34, No. 2, Apr. 1986, pp. 193–202.
2. D. Corne, H.L. Fang, and C. Mellish, "Solving the Module Exam Scheduling Problem with Genetic Algorithms," *Proc. Sixth Int'l Conf. Industrial and Engineering Applications of Artificial Intelligence and Expert Systems*, Gordon and Breach Science Publishers, 1993, pp. 370–373.
3. R. Feldman and M. C. Golumbic, "Constraint Satisfiability Algorithms for Interactive Student Scheduling," *Proc. 11th Int'l Joint Conf. Artificial Intelligence*, Vol. 2, Morgan Kaufmann, San Francisco, 1989, pp. 1010–1016.
4. S. Minton, "Integrating Heuristics for Constraint Satisfaction Problems: A Case Study," *Proc. 11th Nat'l Conf. Artificial Intelligence*, AAAI /The MIT Press, Cambridge, Mass., 1993, pp. 120–126.
5. M. Yoshikawa and S. Wada, "Constraint Satisfaction with a Multi-Dimensional Domain," *Proc. First Int'l Conf. Artificial Intelligence Planning Systems*, Morgan Kaufmann, 1992, pp. 252–259.
6. S. Minton et al., "Minimizing Conflicts: A Heuristic Repair Method for Constraint Satisfaction and Scheduling Problems," *Artificial Intelligence*, Vol. 58, Dec. 1992, pp. 160–205.

Table 1. Data for the school scheduling problems.

Problem	Number of lessons (variables)	Number of time slots (domain size)	Number of classes	Number of teachers [part-timers]	Number of constraints [definitions]	Total penalty	Absence ratio (%)
Kuki92L	806	34	30	60 [9]	24,987 [54]	220,245	6.6
Kuki93L	783	34	30	63 [10]	25,881 [80]	231,204	9.5
Daito93L	633	32	26	69 [29]	26,014 [85]	266,508	33.0
Daito94L	668	34	33	70 [30]	25,796 [90]	305,656	32.1
Kuki92C	806	34	30	60 [9]	–	–	6.6
Daito94C	668	34	33	70 [30]	–	–	32.1
Okou94C	930	33	34	82 [–]	–	–	3.8

A constraint not only has a condition to be satisfied, but also penalty points that a violation of the constraint will incur. A CRP is the same as a *constraint satisfaction problem* (CSP), except that a constraint has a penalty and the CRP's goal is to minimize the total penalty points arising from inconsistent constraints.

A lesson is one variable in a high school scheduling problem. Its domain is the set of all time slots during a week. The problem is to assign a time slot to every lesson while considering various constraints. A lesson's teachers and classes are considered by the conditions of that lesson's constraints.

Scheduling at Okou High School, for example, must handle 34 classes, 82 teachers, 33 weekly time slots, and 930 lessons. The problem involves assigning the 33 time slots to each of 930 lessons, which is equivalent to selecting from $10^{1.412}$ ($=33^{930}$) combinations. Moreover, there are various absolutely required constraints and other subtly preferred constraints.

In our case study, we applied Coastool/Lisp and SchoolMagic to three Japanese high schools, as shown in Table 1. In this table, the absence ratio is

$$\text{absence ratio} = \frac{\sum^{teachers}(\text{number of a teacher's absent time slots})}{(\text{number of teachers}) \times (\text{number of slots})}$$

We first applied Coastool/Lisp to the 1992 and 1993 scheduling problems (Kuki92L and Kuki93L) for Saitama Prefectural Kuki-Hokuyou High School. This mammoth high school offers a great variety of courses, so scheduling usually requires 100 person-days. Our goal in the Kuki92L problem was to satisfy all constraints. The problem included a subtle constraint, Separate 2% Credits, which specifies that a class should take two lessons for a two-credit subject two or more days apart. The Kuki93L problem was similar, but more tightly constrained, because it had a higher absence ratio and more simultaneous lessons.

Second, we applied Coastool/Lisp to the scheduling problems of the Daito Bunka University Dai-ichi High School, a nongovernmental, academic high school with very different problems (Daito93L and Daito94L). Because it has many part-time teachers and one training day without a lesson for each teacher, Daito's absence ratio is very high (33.0% in 1993 and 32.1% in 1994). Moreover, because the high school has many optional courses, it offers several simultaneous lessons with various subjects, teachers, and classes. Taking two lessons with the same subject in a day is unavoidable at Daito. The lessons should be continuous, however, if they are taken in a day. Indeed, the schedule for Daito93L constructed by a teacher has about 80 continuous pairs of the same subject lessons.

Third, after we developed SchoolMagic with Coastool/C++, we applied it to the Kuki92C and Daito94C problems. Because the Coastool/Lisp problem specification differs in a few places from SchoolMagic's, these two specifications differ slightly from Kuki92L and Daito94L. SchoolMagic, for example, handles teacher limits on the numbers of all lessons and continuous lessons in

a day by implementing these constraints as virtual constraints to reduce computational time and memory size.

Finally, we evaluated SchoolMagic at the Kouchi Prefectural Okou High School. As the largest school in Kouchi prefecture, it offers an academic course, a business course, and an art course. The Okou94C problem resembled, but was more difficult than, the Kuki problems. At Okou, where Saturday is an off day every other week, the school offers many continuous and simultaneous lessons, and supervising teachers cannot have a first-period lesson.

Coastool architecture

If we were to apply a traditional production system to scheduling, we would interview human experts to obtain their know-how, model that knowledge, and program it as production rules. However, no straightforward know-how is available for these school scheduling problems. Rather, human experts use trial and error to search for solutions. Therefore, we took a completely different approach in using the Coastool general-purpose CRP solver.

Coastool's constraint-based architecture has a declarative description language and a library of problem-solving methods (see Figure 1). It solves a problem by describing it and selecting a method. The language translates into a common network data structure for constrained problems. Then the selected problem-solving method solves the problem by processing the data structure.

Figure 2 shows a declarative problem description. Lines 1 to 3 define a set of variables, `Lesson`, and a domain set, `Time`. Here, Coastool has modeled simultaneous lessons as one lesson—`Gym1 123a`—that corresponds to three gymnastic lessons for classes 1-1, 1-2, and 1-3.

In lines 6 to 10, a constraint definition defines a set of constraints, named `PartTimerTanaka`. The description specifies that the constraints are defined on all elements of the set `TanakaLesson`. Let `lesson` be a `TanakaLesson` element. A constraint has a variable `lesson`. Let `v` be its value. The condition to be satisfied is that `v` is not a `TanakaAbsence` element, and violating the constraint incurs the 10-point penalty.

Lines 11 to 14 define a CRP (`Timetabling`) that has a set of variables (`Lesson`), with a domain (`Time`), and constraints (`PartTimerTanaka`), and so on. The system predefines the object function `TotalPenalty`.

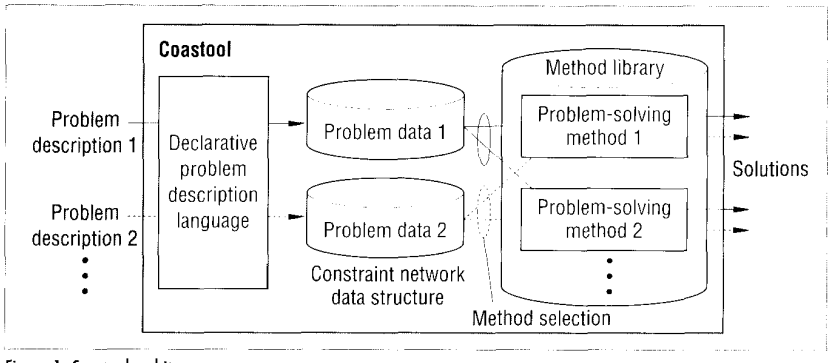

Figure 1. Coastool architecture.

```
1    (define set Lesson
2        Math1 1a Math1 1b... Gym1 123a...)
3    (define set Time Mon1 Mon2... Sat1...)
4    (define set TanakaLesson Gym1 123a Gym1 123b...)
5    (define set TanakaAbsence Mon2 Tue3... Sat1 Sat2)
6    (define constraint PartTimerTanaka
7        :objects ((:set lesson TanakaLesson))
8        :variables ((v lesson))
9        :condition (not (is-a v TanakaAbsence))
10       :penalty 10)
11   (define problem Timetabling
12       :variables ((Lesson Time))
13       :constraints (PartTimerTanaka...)
14       :minimize TotalPenalty)
```

Figure 2. Problem description sample.

In the Coastool architecture, a problem description is declarative and independent of the problem-solving method, which offers several advantages:

- *High productivity for modeling real-world problems.* Because Coastool can solve a problem by declaring "what the problem is," without programming "how to solve it"—that is, without the know-how—it requires minimal information about the problem.
- *High maintainability.* Because a problem description is declarative, the Coastool architecture can maintain a problem by adding, deleting, or modifying a small number of constraints.
- *Wide range of applications.* Coastool can select a problem-solving method according to a particular feature of a target problem. Moreover, changing a method does not require modification of the problem description.

Consequently, separating the problem description from the problem-solving method results in high productivity, high maintainability, and a wide coverage range. However, these advantages rely on the efficiency of the problem-solving method.

Problem-solving method

How many combinations can we examine in a reasonable computational time? Even if we can examine 10^{100} combinations, that's only $1/10^{1,310}$ percent of the whole search space. Therefore, an optimization method has little chance of finding a good solution in a biased random walk. From these considerations we concluded that we would need to begin optimization from a very high-quality solution.

Background. Most general-purpose methods for constrained problems fall into the categories of *consistency algorithms, backtracking,* and *optimization.* Consistency algorithms and backtracking suffer from thrashing for large-scale problems. In our study, fast backtracking algo-

```
1  Begin Really-Full-Lookahead Greedy Algorithm
2    Let Vars = set of all variables, ExVars = empty set,
         Var = NULL.
3    While Vars is not empty:
4        Let DsStore = copy of all domains of Vars.
5        Enforce all domains of Vars to be arc-consistent
             with Var.
6        If there is a variable V with an empty domain in
             Vars,
7          then remove V from Vars and add V into ExVars;
8               restore all domains of Vars from DsStore.
9          else Let Var = a variable in Vars;
10              Let Value = a value in Var's domain;
11              Assign Value to Var and remove Var from
                  Vars.
12       end if
13   end while
14   While ExVars is not empty:
15       Let Var = a variable in ExVars.
16       Let Value =  one of the least penalty values
                      in Var's original domain.
17       Assign Value to Var and remove Var from ExVars.
18   end while
19 End Really-Full-Lookahead Greedy Algorithm
```

Figure 3. Really-Full-Lookahead greedy algorithm.

rithms—*Forward-Checking Backtracking* and *Iterative Broadening*[2]—assigned at most 20% of the lessons in a one-week run. Consequently, in this article, we will focus on optimization methods.

Most optimization methods involve randomly walking around in the problem search space under a bias to find an optimal solution. The bias strength affects computational time and optimality: A stronger bias increases efficiency but decreases optimality.

One of the most common general-purpose optimization methods is *Simulated Annealing*.[3] Because SA is a weakly biased method, it finds an optimal solution independently of an initial solution. Its computational time is too long, however, for practical use in high school scheduling.

Steve Minton and his colleagues solved a large-scale, loosely constrained *Million Queens Problem* using a *greedy* algorithm for initialization and the *Min-Conflicts Hill-Climbing* method for optimization.[4] Because MCHC is one of the most strongly biased optimization methods, it solves a large-scale CRP efficiently. As Minton reported, however, an inappropriate initial solution falls into a local optimum for a tightly constrained problem.

Recently, Ron Musick and Steve Russell showed the theoretical importance of initialization.[5] Namely, when the distance between an initial solution and an optimal solution exceeds a border value, the computational time grows exponentially.

Proposed method. We have focused on developing a high-quality initialization method and combining it with a strongly biased optimization one. The proposed method is a combined one, involving a novel initialization algorithm, *Really-Full-Lookahead Greedy* (RFLG), and the MCHC strongly biased optimization algorithm.

Figure 3 shows the two-step RFLG algorithm. The first step (lines 3 to 13) consistently assigns a value (a time slot during a week) to each variable (lesson) using an arc-consistency algorithm. If the algorithm fails to find a consistent value for a variable, it excludes the variable. A common greedy algorithm in the second step (lines 14 to 18) will instantiate the excluded variable with an inconsistent value.

At first, `Vars` is initialized by the set of all variables (line 2). The first step loops the removal of a variable `V` or `Var` from `Vars` until `Vars` is empty, so the number of iterations equals the number of all variables. In the iteration, an arc-consistency algorithm removes candidate values of variables in `Vars` that cannot satisfy a constraint from their domains (line 5). Once it removes some values, it checks all constraints associated with the variable to propagate the removal from the variable to another, until no more values can be removed.

For example, let `Gym1-123a` and `Gym1-123b` in Figure 2 be continuous. Because Tanaka is absent on `Mon2`, the algorithm removes the value `Mon2` from the domains of variables `Gym1-123a` and `Gym1-123b`. Furthermore, because `Gym1-123a` and `Gym1-123b` must be continuous and `Gym1-123b` cannot be `Mon2`, `Gym1-123a` cannot be `Mon1`, and the algorithm also removes `Mon1` from the domain of `Gym1-123a`. Now, the `Gym1-123a` domain is `{Mon3, Mon4, ...}`. So, the step selects `Gym1-123a` as a variable `Var` (line 9) and `Mon3` as a value `Value` from the domain (line 10) and assigns the variable (line 11).

In the next iteration, the arc consistency algorithm forces the domain of `Gym1-123b` to be `{Mon4}`, because `Gym1-123a` is already assigned to `Mon3` and they must be continuous. Therefore, it assigns the consistent value `Mon4` to `Gym1-123b`. Here, the algorithm uses the *Most-Constrained-Variable-First* variable selection heuristic in line 9 and the *Least-Constraining-Value-First* value selection heuristic in line 10. Ties are broken randomly in the selections.

The first step never instantiates an inconsistent assignment. If a variable `V` appears with an empty domain in the arc consistency algorithm (line 6), it means there is no consistent value for `V`. In this case, the step suspends assigning a value to `V`, excludes it from `Vars`, and stores it in `ExVars` (line 7) for use in the second step. That step restores the domains of `Vars` as the domains in `DsStore` before forcing arc consistency (line 8). Then it continues to instantiate other consistent assignments.

If there are no more consistent assignments, the RFLG algorithm proceeds to the second step (lines 14–18). The step is a common greedy algorithm that assigns inconsistent but least-penalty values on the basis of local evaluation to the excluded variables. Here, the algorithm randomly selects `Var` (line 15), and randomly breaks ties in the value selection (line 16).

In the first step of the RFLG algorithm, the use of the arc-consistency algorithm avoids assigning a value that is consistent with already assigned variables but that would cause later inconsistency with an *As-Yet-Uninstantiated* variable. For example, if we use a simple greedy algorithm, it may assign `Mon1` to `Gym1-123a`, because it has no immediate violation until `Gym1-123b` is instantiated. The RFLG algorithm removes `Mon1` from the domain of `Gym1-123a` by arc consistency before selecting a value for the variable.

The use of arc consistency and variable/value selection heuristics is common in backtracking algorithms for constraint satisfaction problems. In backtracking algorithms, this use reduces the number of backtracks: arc consistency prunes inconsistent

branches in a search tree and selection heuristics give priority to a branch with a high likelihood of finding a solution. With the RFLG algorithm, this use reduces the number of excluded (inconsistent) variables for the same reason. Here, the RFLG algorithm uses arc consistency for constraint relaxation, not for constraint satisfaction.

In general, a scheduling algorithm can abandon inconsistent values for satisfaction, but must consider them for relaxation. Our method assigns inconsistent values in the second step of the RFLG algorithm and in the MCHC algorithm. Consequently, the RFLG algorithm generates a high-quality initial assignment, employing several techniques commonly used in backtracking algorithms.

Bernard Nadel unified several backtracking algorithms as combinations of pure *tree search* and arc consistency.[6] From this viewpoint, we can consider the RFLG algorithm as a combination of pure generation and arc consistency. It never backtracks but corresponds to the *Really-Full-Lookahead Backtracking* algorithm, because both use full arc consistency. Moreover, we can extend this idea to develop several-generation algorithms with partial arc consistency, such as the *Forward-Checking Greedy* algorithm.

Coastool also has a program-synthesizing facility that produces an efficient arc-consistency program for a given constraint relaxation problem. The problem-description language compiles a condition form for a constraint definition into an efficient constraint propagation procedure, which will be called by a generic arc-consistency program.[7]

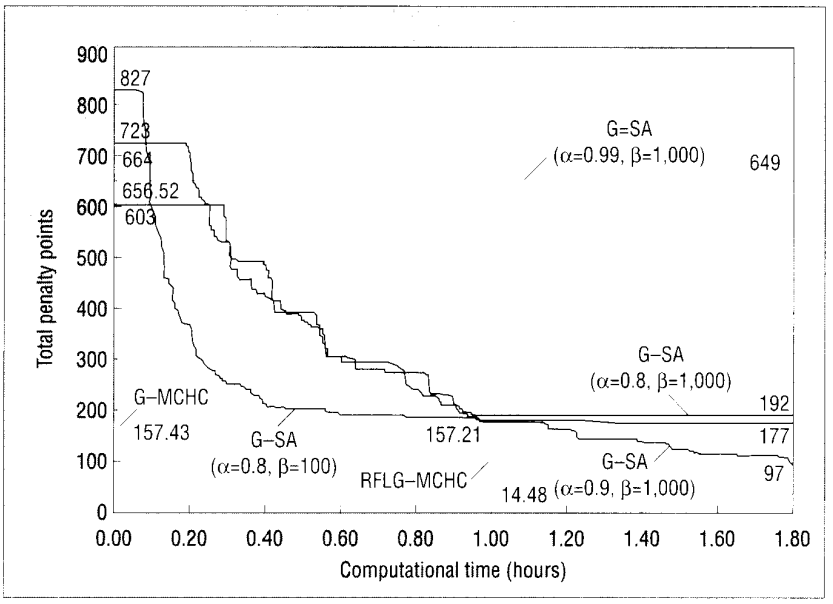

Figure 4. Experimental results for the Kuki92L problem: In SA, α is the cooling ratio and β is the number of iterations at the same temperature.

Evaluation of Coastool/Lisp

We applied Coastool/Lisp to two high schools, Kuki-Hokuyou and Daito Bunka.

Kuki-Hokuyou. At this school, 10 teachers usually work on scheduling for 10 days. We interviewed the scheduling leader to determine the data and requirements for Kuki92L. We then constructed a schedule with Coastool and asked the school's key scheduling person to evaluate that schedule.

Coastool architecture. The interview took three hours and describing the problem took three person-days. The description includes 1,500 lines of data definitions and 500 lines of problem specification. There were 54 constraint definitions (see Table 1). The results show that the productivity for modeling real-world problems is considerably higher than with production system approaches.

Three difficulties became clear:

- Set-definition language structures are too simple to describe various data conveniently.
- Lisp and the large constraint network require a large amount of memory.
- Modeling a global constraint in a CRP produces a huge description and too many constraint instances for easy handling.

A global constraint is either a constraint with all variables or a conceptual constraint in which the combinations of related variables are determined dynamically according to the assignments. For example, consider the constraint of teacher limits on the total number of lessons in a day. If Tanaka has n lessons during a week and the limit in a day is l, we model the constraint as ${}_nC_{l+1}$ constraint instances: every combination of $l+1$ lessons from n forces us to assign at least one of the $l+1$ lessons to a different day from the others. The problem-solving method specially considered these constraints when breaking ties during value selection. The new C++ version of Coastool has solved these problems.

Problem-solving method. Figure 4 shows the transition of total penalties incurred by the proposed method (RFLG-MCHC), the combination of the greedy algorithm and MCHC (G-MCHC), and the combination of the greedy algorithm and SA (G-SA). The figure plots only optimization phases. Because the RFLG algorithm takes about one hour to initialize, the RFLG-MCHC curve begins at about that point. The G-MCHC method falls into a local optimum in several minutes. The scheduling teacher found that G-MCHC achieved less than a 70% completion degree—that is, the solution had a lower quality than 70% of the human scheduling solutions. We applied the G-SA method with several values for cooling parameters, but it was too slow to construct a useful schedule, requiring at least three days to reach the 90% completion point.

Although the RFLG algorithm required about one hour to complete, our approach generated a high-quality initial assignment, which MCHC refined and used to construct an almost perfect schedule (better than 95%). Its best solution had only one inconsistent constraint (`SeparateTwoCredits`) in 60 trials. Consequently, the method constructs an almost perfect schedule reasonably quickly. It is considerably faster than G-SA and produces a considerably better schedule than G-MCHC. However, it could not sufficiently satisfy the two global constraints—teacher limits on the number of all (and continuous) lessons in a day—that we discussed earlier. It failed to implement them as CRP constraints and merely handled them as preferences in tie-breaks.

We also considered other combinations of initialization and optimization. RFLG-SA doesn't work well, because SA at first walks from an initial assignment to much worse solutions. Figure 4 suggests that we apply

```
class TeacherAbsence :public Constraint {
        Teacher *teacher;
        TimeSlotVariable *variable;
    public:
        TeacherAbsence (Teacher *t, Lesson *l)
            :teacher(t), variable(l->timeSlot()) {} ;
        virtual int doCheck() const {
            return(isTeacherAbsentAt (*teacher, variable->
                value())) ;
        } ;
        virtual float penalty() const {return 10.0 ;};
    ...
}
```

Figure 5. Constraint definition with Coastool/C++.

MCHC on a solution in the middle of SA cooling, but such an approach produced an unacceptable local optimum that was far from a global optimum.

Total performance. As a whole, describing the Kuki92L problem required only three person-days, and constructing an almost perfect schedule took one hour. We had to modify the problem description several times. Using Coastool thus reduced the cost for scheduling from 100 person-days to less than one person-week.

Maintainability. To confirm these successful results, we returned to the same school the next year to construct a new schedule (Kuki93L). Modifying the problem description from Kuki92L to Kuki93L took just 1.5 person-days and merely involved adding, deleting, and rewriting a few constraints. Most of the maintenance was data maintenance.

In addition, the problem data or requirements changed several times in the middle of the scheduling task. For example, teachers in charge of lessons were changed according to their assignability. Minor changes required only about ten minutes each, which shows Coastool's high maintainability.

Partial arc consistency. The computation was also successful and useful. Constructing a schedule at 90% (76 penalty points) took about 2.4 hours. However, because the RFLG algorithm repeatedly uses an arc-consistency algorithm, it required 2.2 times as much computational time as in Kuki92L, even though the problems were for the same school.

Although the result was successful, we tried to decrease the computational time, using a partial arc-consistency algorithm. That is, we used arc consistency to process only important variables and constraints, assigning the other variables in the second step of the RFLG algorithm. We considered the other constraints in value selection. As a result, our method constructed a schedule at 70% (257 points) in 26.2 minutes. The RFLG algorithm affords the flexibility to smoothly tune the trade-off between efficiency and optimality.

Daito Bunka. We applied Coastool/Lisp to the scheduling for a different kind of high school. At Daito, the teacher in charge of schedules usually uses the Shiraku scheduling software[8] and works on the problem for 12 days. We converted Shiraku's input files into a Coastool problem description.

Phased approach. Because Daito's problems are much more difficult than Kuki's, Coastool needed 2.8 hours to achieve a 70% complete schedule with Daito93L. Our method fell into an unacceptable local optimum.

We tried a phased approach following the teacher's expert technique. That is, we assigned important lessons first and had a nonexpert person modify them; then we assigned the others and had those modified. The first assignment required 37.6 minutes, and modification took 20 minutes. The second assignment took another 91.5 minutes, and final modification took three hours. A nonexpert person performed the modification using a graphical user interface and produced a 90% complete schedule. Coastool thus reduced the total cost from 12 days by an expert to four days with a nonexpert. These results were similar to the next year's Daito94L problem.

The human modification repeated a kind of local search that finds a plan to repair several lessons to satisfy constraints, according to constraint priority. All absolute requirements must be satisfied. Because the plan temporarily passes higher penalty solutions, it does not lend itself to simulation with a hill-climbing method. Automating such planning is interesting work for the future.

These evaluations show that Coastool/Lisp is efficient as a base for practical applications in high school scheduling problems.

Coastool/C++

To apply the effective techniques widely, we reimplemented Coastool with C++ and developed an automatic high school scheduling system, SchoolMagic, that runs on Microsoft Windows.

Our evaluation of Coastool/Lisp clarified several difficulties:

- The problem-description language was not powerful enough and required considerable memory.
- The problem-solving method was not fast enough and couldn't handle global constraints efficiently.

To overcome these difficulties, we developed a new version of Coastool using C++ rather than Lisp. Its C++ class library provides general-purpose C++ programming classes (*containers* and *iterators*), powerful classes for *sets* (such as `Domain`, `UniversalSet`, `EnumerativeSet`, and `ContinuousSet`), basic classes for CRPs (such as `Variable`, `Constraint`, and backup information), and assignment routines (such as `RFLG` and `MCHC`).

This C++ library approach solves the difficulty of expressiveness. Users can develop an application using the library as they desire. They can develop variables and constraints for the application by defining derived classes that are inherited from base classes `Variable` and `Constraint` and by customizing the standard interface (member functions) of the base classes.

Figure 5 shows an example of a constraint definition. In the figure, standard member functions `doCheck` and `penalty` are customized. The C++ library approach helps solve the difficulties of speed and memory size: C++ is faster than Lisp and doesn't require Lisp language kernel or a lot of library functions. With the C++ library, only required functions link to the application system.

The most important feature introduced by Coastool/C++ is the implementation of virtual constraints for handling global constraints. In a *static* constraint network data structure used by Coastool/Lisp, *data pointers* associate variables and constraints with each other. The pointer network consumes considerable memory and computational time. Kuki92L has 95,588 pointers between 806 variables and 24,987 constraints. A variable associates with an average of 59 constraints, and each must be checked when the variable's value changes.

For example, consider a global constraint that no teacher can have more than one lesson at a time. Figure 6 illustrates the static constraint network with this global constraint. Because Tanaka has five lessons in this figure and every combination of two lessons must be assigned with different time slots, there are 10 static constraints. Four constraints are associated with each lesson. These constraints and the network data structure consume a huge amount of memory. Moreover, they require many iterations over the network. For example, when the value of `Gym2-1a` is modified, the four constraints associated with the variable must be checked.

In general, the global constraint is implemented by ${}_nC_2$ static constraints with every combination of two lessons, where the teacher has n lessons. Each lesson is associated with $n-1$ static constraints that must be checked when its value is modified. Because a teacher usually has about 17 lessons during a week, he or she has ${}_{17}C_2 = 136$ static constraints, 16 with each lesson. In total, Kuki92L has 60 teachers and 8,890 ($= \sum^{teachers}\{{}_nC_2\}$) static constraints.

Using Coastool/C++ lets users implement the global constraint as only one virtual constraint, which creates constraint instances dynamically only when the condition is violated.

Figure 7 shows the constraint network with a virtual constraint. Every lesson shares a virtual constraint. The application has a weekly schedule data structure (Tanaka's weekly schedule) to keep lists of assigned lessons at each time slot during a week. The algorithm customizes a standard member function `doTryValue` of class `Variable` to keep the weekly schedule when an assignment changes. If `Tue2` is assigned to `Gym2-1a`, the algorithm stores `Gym2-1a` in the schedule at `Tue2`. It then automatically calls another member function `recheck` of the virtual constraint to check constraint violation. It checks Tanaka's weekly schedule to find out if there is another lesson at `Tue2`. Because there is another lesson `Gym1-123a` at `Tue2`, Coastool/C++ instantiates a dynamic constraint with `Gym2-1a` and `Gym1-123a` (Figure 7). When the value will be modified again, the dynamic constraint will be removed automatically.

As the example shows, the virtual constraint replaces $\sum\{{}_nC_2\}(= 8{,}890)$ static constraints with one virtual constraint and only the violated dynamic constraints, using an application-dependent data structure. Moreover, it checks only the teacher schedule rather than the $n-1(\approx 16)$ constraints associated with a lesson.

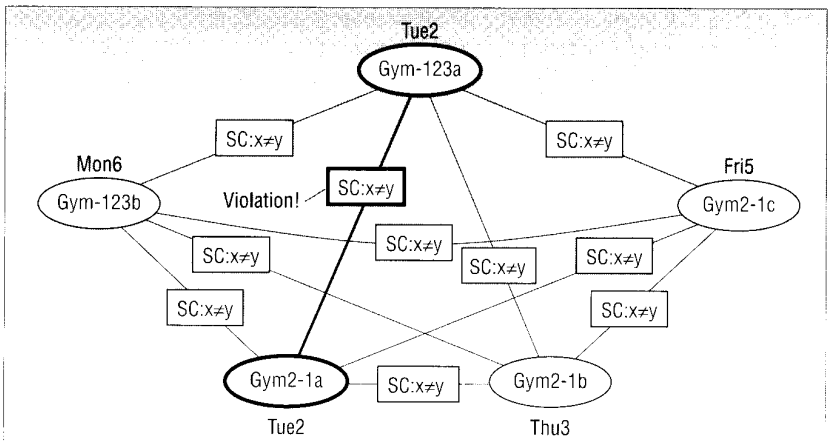

Figure 6. A static constraint network in Coastool/Lisp.

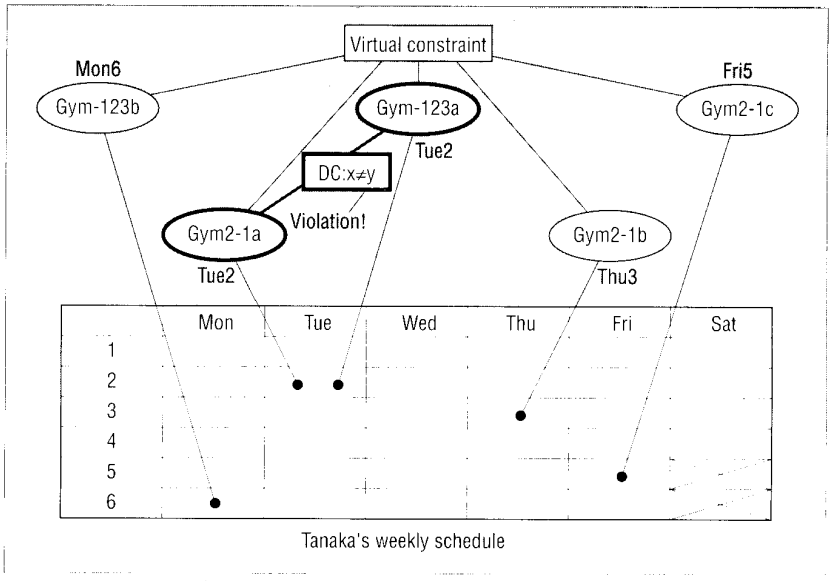

Figure 7. A constraint network with a virtual constraint in Coastool/C++.

A virtual constraint is effective for *constraint propagation* (arc consistency) as well as constraint checking. An arc-consistency algorithm repeats constraint propagation from variable to variable, removing values that cannot satisfy the constraint from their domains, until no more values are available for removal. When some values are removed from the domain of a variable, all constraints associated with the variable must be propagated. For example, if we remove some time slots from `Gym2-1a`'s domain in Figure 6, four static constraints associated with it must be propagated. The propagation of a constraint from `Gym2-1a` to `Gym2-1b` is as follows:

> For each value y in the `Gym2-1b` domain, examine whether there exists a value x in the `Gym2-1a` domain such that the constraint $x \neq y$ is satisfied or not. If no such candidate exists, there is no possibility of y satisfying the constraint. Therefore, remove y from `Gym2-1b`.

In general, this procedure requires $O(d^2)$ time, where d is the domain size. However, we can replace it with the following $O(1)$ procedure, on the basis of the meaning of the $x \neq y$ constraint:[6]

> Check whether the `Gym2-1a` domain has only one candidate or not. If it has no more than one candidate, remove the candidate from the domain of `Gym2-1b`.

This is correct because of a constraint's feature that if `Gym2-1a` has two candidates

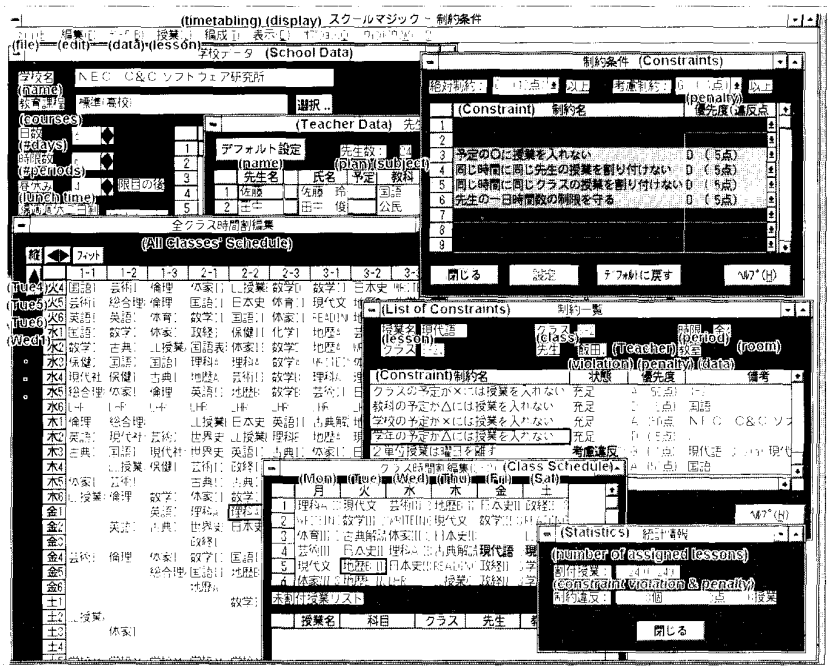

Figure 8. SchoolMagic display image.

Table 2. Comparison between Coastool/Lisp and SchoolMagic.

System/problem	Coastool/Lisp Kuki92L	SchoolMagic (C++) Kuki92C	
Constraint implementation type	Static	Static	Virtual and dynamic
# of static constraints	24,987	30,671	5,144
# of static constraint definitions	54	37	26
# of virtual constraints	0	0	11
Initialization time (sec.)	3,780	2,547	64
Optimization time (sec./2,000 iterations)	77	51	37
Total time (sec.)	3,857	2,624	101
Number of lines of a static constraint definition	9.3	54.0	54.0
Number of lines of a virtual constraint definition	–	–	413.4

or more, no value can be removed from the domain of `Gym2-1b`. For example, if `Gym2-1a` has two candidates, `Mon1` and `Mon2`, `Gym2-1b` can be `Mon1` with `Gym2-1a`'s `Mon2` value. `Mon2` is also possible with `Mon1`, and all the other values are obviously possible. This feature reduces the computational time from $O(d^2)$ to $O(1)$, a speedup that the program synthesizer of Coastool/Lisp had already accomplished.

Now, consider propagating four constraints from `Gym2-1a` to all the other lessons. Coastool/Lisp's arc-consistency algorithm repeats the constraint propagation four times with the other four lessons. In general, it repeats it $n - 1$ times, where the teacher has n lessons, so the computational time is $O(n)$. We can replace it, however, with the following procedure.

Check whether the `Gym2-1a` domain has only one candidate or not. If it has no more than one candidate, remove the candidate from the domains of all the other lessons (`Gym2-1b`, `Gym2-1c`, `Gym1-123a`, and `Gym1-123a`.

Although the worst case (when the `Gym2-1a`'s domain is unique) computational time is $O(n)$, it replaces $n - 1$ checks of the uniqueness with one check. Moreover, considering that the `Gym2-1a`'s domain becomes unique at most once during an arc-consistency process (and during execution of the RFLG algorithm), the average computational time is close to $O(1)$ of the best case (when the domain is not unique). A programmer defines this procedure as a standard member function `reviseFrom` of the virtual constraint in Figure 7. As the example shows, the virtual constraint greatly speeds the arc-consistency process, on the basis of the meaning of the constraint, without expanding a combinatorial constraint network.

Consequently, the virtual constraints of Coastool/C++ have solved the remaining three difficulties of Coastool/Lisp—memory, computational time, and global constraint problems. Replacing several combinatorial constraints and their network pointers with only one virtual constraint and the violated dynamic constraints solves the memory problem. This simplification of the constraint network also solves the speed problem, eliminating many iterations over the network. Moreover, virtual constraints accelerate constraint propagation, on the basis of the meaning of the constraints.

Coastool/C++ uses virtual constraints to implement the two global constraints that Coastool/Lisp could not implement—teacher limits on the number of all lessons and continuous lessons in a day. For example, let a teacher have n lessons with a limit of four lessons in a day. Assigning five lessons in a day violates this global constraint. However, it's impossible to predict which combination of five lessons from the n lessons will meet in a day. Therefore, Coastool/Lisp must provide a static constraint network with ${}_nC_5$ constraints with five lessons of the teacher, forcing assignment of at least one of the lessons to a different day from the others.

Because a teacher usually has about 17 lessons, there must be ${}_{17}C_5 (= 6{,}188)$ constraints with every teacher. In all, there are about 370,000 ($\approx \sum^{teachers} \{{}_nC_5\}$) constraints. Coastool/C++ can efficiently implement this global constraint as a virtual constraint. One virtual constraint and only violated dynamic constraints replace the ${}_nC_5$ constraints. As in Figure 7, Coastool/C++ checks the virtual constraint using the teacher's weekly schedule. The constraint propagation procedure is a bit complicated. It checks whether the day of all candidate time slots in a lesson's domain is unique or not. If unique, it stores the lesson in that day's list. If the size of the list is four, all the teacher's other lessons except those four will not be in that day. Coastool will remove all that day's time slots from the other lessons.

SchoolMagic outline

As Figure 8 shows, SchoolMagic is a multiwindow system. Designed for teachers, it

provides data maintenance windows for entering and editing data, and scheduling windows for showing the current, interactively constructed schedule.

Teachers generally cannot describe constraints in predicate calculus, so SchoolMagic provides 37 kinds of constraints based on our experience with many high schools. We have generalized and parameterized most constraints in the Kuki and Daito problems. For example, we generalized constraint `PartTimerTanaka` described in Figure 2 as the `TeacherAbsence` constraint in Figure 5. Teachers and their absent time slots given by the data maintenance windows set the parameters for this constraint.

The generalized constraints appear in natural language (Japanese) on a window, with users specifying the penalty points. With another facility, users describe new constraints of a certain kind—for example, "gymnastic and mathematics should not be continuous." In this case, teachers use menu selections to specify and set the parameters for the subjects to be constrained (gymnastics and mathematics) and their relations (continuous).

SchoolMagic scheduling windows provide several facilities for scheduling. Explanation facilities show several kinds of schedules (such as a weekly schedule of each class, teacher, and room, and a schedule with all classes, teachers, and rooms), list information about constraints associated with a lesson, and show candidate time slots available for a lesson (or candidate lessons for a time slot). Editing facilities let users assign or unassign lessons, modify (move) the assignment (time slot) of a lesson, and fix or release the assignments on the windows.

Among SchoolMagic's automatic scheduling facilities, an automatic assignment facility provides the proposed problem-solving method. An automatic refinement facility provides the MCHC method for use in interactive rescheduling. Because the automatic refinement moves only released lessons, it obeys user directions from fix-and-release operations. Users therefore can interactively modify a schedule through editing and automatic refinement. Indeed, this interactive rescheduling greatly simplifies the human modification required in Coastool/Lisp evaluation. A user can specify lessons for automatic assignment or refinement. This facility supports the phased approach used in Coastool/Lisp evaluation. Moreover, SchoolMagic provides a batch-scheduling facility that repeats the automatic assignment and file save with different random number seeds. After the batch process, a user can select the best schedule for the given data and modify it through interactive rescheduling.

Evaluation

Table 2 compares Coastool/Lisp (Kuki92L) and SchoolMagic (Kuki92C) on Coastool/C++. We implemented SchoolMagic in two ways: with and without virtual constraints. The comparison shows several dramatic effects of Coastool/C++.

- Virtual constraints have reduced by one sixth the number of constraints. Although comparing memory consumption between Lisp and C++ is difficult, Kuki92C on SchoolMagic requires only about one tenth of the memory required by Kuki92L on Coastool/Lisp.
- SchoolMagic decreased the computation time to about one fortieth of Coastool/Lisp. Using virtual constraints decreased the initialization time by about one fortieth, even if compared on the same Coastool/C++ with the same problem, Kuki92C. Virtual constraints eliminated many arc-consistency iterations.
- Because of the lack of a declarative problem-description language, Coastool/C++ increased the description length for constraint definitions, especially for virtual constraints. This is a remaining problem for Coastool/C++.

SCHOOLMAGIC PROVIDES A BATCH-SCHEDULING FACILITY THAT REPEATS THE AUTOMATIC ASSIGNMENT AND FILE SAVE WITH DIFFERENT RANDOM NUMBER SEEDS.

Figure 9 (next page) shows the transition of total penalties with the proposed RFLG-MCHC method and previous methods G-MCHC and G-SA with Kuki92C (Coastool/C++). For Coastool/Lisp (see Figure 4), the RFLG initialization algorithm took about one hour, making it considerably slower than the G-MCHC method. On Coastool/C++ (Figure 9), the RFLG-MCHC method took only double the computational time of the previous G-MCHC method, because virtual constraints sped up the RFLG algorithm 40-fold. Compared to G-SA, the RFLG-MCHC method is much faster and better.

We applied SchoolMagic to three high school scheduling problems, Kuki92C, Daito94C, and Okou94C. With every high school, the data input required less than two person-days. Moreover, the automatic assignment facility took six minutes or less on a Pentium PC. Human modification required a few minutes with Kuki92C, an hour with Okou94C, and a few hours with Daito94C. Even if we took the backward modification of input data and requirements into account, one day was sufficient for scheduling after the first input data were given.

Okou94C was more difficult than Kuki92C because that school has so many continuous and simultaneous lessons. Here, the phased approach was very effective. All lessons were assigned at once by the problem-solving method. In about 70% of the lessons, the continuous and simultaneous lessons were unassigned, and the partial schedule was repaired by the interactive rescheduling. Again, the problem-solving method assigned the other lessons. Finally, we used interactive rescheduling to modify the entire schedule to get an almost perfect schedule.

Why were 70% of the lessons assigned and unassigned later? When we assigned only the other lessons first, we could not grow our initial schedule into a complete, good-quality schedule because the RFLG algorithm uses arc consistency with as-yet-uninstantiated variables to get a good initialization. The most important point of the problem-solving method is considering constraints with uninstantiated variables.

SchoolMagic has global constraints implemented as virtual constraints on Coastool/C++. This feature was very useful in Kuki92C, because we could construct a schedule that satisfied all teacher limits on the numbers of all lessons and continuous lessons in a day. Okou94C satisfied most of the limits, but with Daito94C, we could not use the global constraints because the problem was too difficult to obey them. Moreover, specifying them meant violating other, more important constraints. Because the problem-

solving method relies on constraint satisfaction techniques such as arc consistency, having too many constraints may actually degrade the schedule.

By constructing an applicable schedule in less than three person-days for each high school, SchoolMagic reduced the original cost in Kuki-Hokuyou by 1/33rd and by 1/50th in Okou (the largest school studied). In Daito94C, SchoolMagic decreased the total cost to one-third, down from 12 person-days with commercial software. The scheduling cost after data input was 1/10th that when using software.

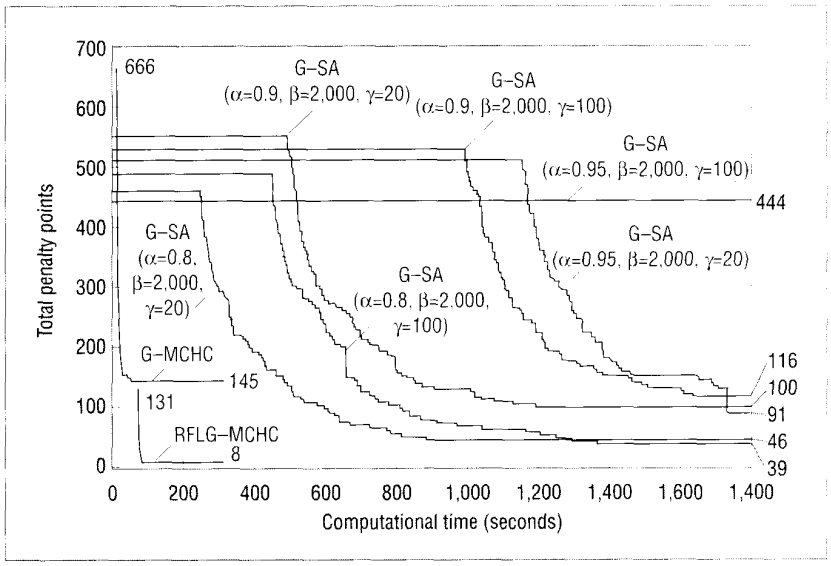

Figure 9. Experimental results for problem Kuki92C: In SA, α is the cooling ratio, β is the number of repeating times at the same temperature, and γ is the initial temperature.

OUR FUTURE WORK INCLUDES devising a declarative problem-description language based on the C++ language. We would also like to enable the problem-solving method to escape from local optima and give it an automatic phased approach. We are looking as well for wider applications of Coastool/C++ for scheduling and assignment problems.

Acknowledgments

We thank Masahiro Yamamoto of the NEC Corporation for encouraging us to pursue this research. We also thank Kin-ya Ohuchi, Mikio Nakabayashi, and Mitsuhiro Masuda of Kuki-Hokuyou High School, Yosuke Kamine of Daito Bunka University Dai-Ichi High School, and Nobuomi Komatsu and Mikihide Imai of Okou High School for their cooperation in the evaluation.

References

1. M. Yoshikawa et al., "A Constraint-Based Approach to High School Scheduling Problems: A Case Study," *Proc. 12th Nat'l Conf. Artificial Intelligence*, Vol. 2, AAAI/MIT Press, Cambridge, Mass., 1994, pp. 1111–1116.

2. M.L. Ginsberg and W.D. Harvey, "Iterative Broadening," *Artificial Intelligence*, Vol. 55, Nos. 2–3, 1992, pp. 367–383.

3. D.S. Johnson et al., "Optimization by Simulated Annealing: Experimental Evaluation; Part II, Graph Coloring and Number Partitioning," *Operations Research*, Vol. 39, No. 3, June 1991, pp. 378–406.

4. S. Minton et al., "Minimizing Conflicts: A Heuristic Repair Method for Constraint Satisfaction and Scheduling Problems," *Artificial Intelligence*, Vol. 58, Nos. 1–3, Dec. 1992, pp. 160–205.

5. R. Musick and S. Russell, "How Long Will It Take?," *Proc. 10th Nat'l Conf. Artificial Intelligence*, AAAI/MIT Press, 1992, pp. 466–471.

6. B. Nadel, "Tree Search and Arc Consistency in Constraint-Satisfaction Algorithms," in *Search in Artificial Intelligence*, L. Kanal and V. Kumar, eds., Springer Verlag, New York, 1988, pp. 287–342.

7. M. Yoshikawa and S. Wada, "Constraint Satisfaction with a Multi-Dimensional Domain," *Proc. First Int'l Conf. Artificial Intelligence Planning Systems*, Morgan Kaufmann, San Francisco, Calif., 1992, pp. 252–259.

8. *Shiraku II Ver. 2.1 Users Manual* (in Japanese), Yasukawa Information Systems Corp., Saitama, Japan, 1992.

Masazumi Yoshikawa is an assistant manager of the Software Research Laboratory at NEC Corporation's C&C Research Laboratories. For the last five years, he has been the technical project leader for Coastool and its application for many scheduling and assignment problems. He is currently a visiting scientist at the University of Southern California's Information Sciences Institute. His research interests include optimization, machine learning, specification, and program synthesis. He received an ME in information processing in 1986 and a BS in physics in 1984, both from Nagoya University. He can be reached at USC/ISI, 4676 Admiralty Way, Marina del Rey, CA 90292-6695; yosikawa@isi.edu.

Kazuya Kaneko is a computer scientist at NEC's Software Research Laboratory. His research interests include scheduling expert systems and combinatorial optimization. He received an ME and a BE in electrical and communication engineering from Waseda University in 1993 and 1991. He can be reached at C&C Research Laboratories, NEC Corp., Miyazaki 4-1-1, Miyamae-ku, Kawasaki, Kanagawa 216 Japan; neko@swl.cl.nec.co.jp.

Toru Yamanouchi is a research manager at NEC's Software Research Laboratory. His research interests include artificial intelligence and knowledge-based software engineering. He received a BE in control engineering from the Tokyo Institute of Technology in 1980, an MS in computer science from the University of California, Los Angeles, in 1982, and an ME in information processing from the Tokyo Institute of Technology in 1983. He was a visiting scientist at Carnegie Mellon University during the 1990–1991 academic year. He can be reached at C&C Research Laboratories, NEC Corp., Miyazaki 4-1-1, Miyamae-ku, Kawasaki, Kanagawa 216 Japan; yamano@swl.cl.nec.co.jp.

Masanobu Watanabe is a senior manager of NEC's Software Research Laboratory, where his activities focus on software architecture and its automated software design and development environments, including scheduling systems, business applications, built-in real-time systems, mobile computing systems, and multimedia systems. He received his BE, ME, and DE in applied mathematics and physics engineering from Kyoto University in 1975, 1977, and 1983. He can be reached at the NEC Corp., 4-1-1, Miyazaki, Miyamae-ku, Kawasaki, Kanagawa 216 Japan; watanabe@swl.cl.nec.co.jp.